

ARCHITECTURE, CAPABILITIES & ECOSYSTEM SPECIFICATION

The Complete System Dossier: From Concept to Codebase

An exhaustive technical analysis of Axon AI Studio — the local-first desktop platform where AI transcends conversational textboxes to inhabit an embodied, spatial 3D virtual office campus of 208 specialized coworkers.

208

COWORKERS SEATED

39

MCP CONNECTORS

31

LICENSED SKILLS

0

VULNERABILITIES (AUDIT)



Table of Contents

1. Executive Overview & System Philosophy	Ch. 1
1.1 Paradigm Shift: The "AI Has An Office Now" Thesis	Overview
1.2 Core Architectural Principles & Zero Cloud Lock-In	Principles
1.3 Key Technical Metrics & Stack Specification	Stack
2. What Axon Does: Core Capabilities & User Experience	Ch. 2
2.1 The 3D Virtual Campus & Procedural Low-Poly Simulation	Spatial
2.2 The Split-Screen Coworker Workspace	Workspace
2.3 Multi-Provider AI Engine & BYO-Key Protocols	Inference
2.4 Human-in-the-Loop Safety, Diff Approvals & Coworker Undo	Safety
2.5 Local Document Intelligence & Process-Isolated RAG	Retrieval
2.6 Model Context Protocol (MCP) Integration Engine	Connectors
2.7 Transparency, Cost Metering & Governance	Auditing
3. What Axon Contains: Structural & Inventory Blueprint	Ch. 3
3.1 Repository Anatomy & Complete Source Code Hierarchy	Directory
3.2 The 9 Core Commons Coworkers	Roster
3.3 The 199 Domain Specialists across 22 Department Groups	Specialists
3.4 The 39 Catalog Connectors (Model Context Protocol)	Integrations
3.5 The 895 Ingested Skills & Synthesis Pipeline	Skills
3.6 Design System Architecture & UI Primitives	Design Tokens
3.7 Storage, State Schemas & DPAPI Vault	Persistence
4. What All We Have Done: Chronological Engineering Journey	Ch. 4
4.1 Evolutionary Milestones (Wave 0 through Wave 7)	History
4.2 Major Engineering Breakdowns & Solved Hurdles	Engineering
4.3 Dependency Hardening & Vulnerability Remediation (19 → 0)	Hardening
4.4 Testing, E2E Smoke Validation & Benchmarks	Verification
4.5 Marketing Platform & Launch Motion Production	Media
5. What Axon Can Do: Operational Scenarios & Real-World Workflows	Ch. 5
5.1 Multi-Agent Collaborative Full-Stack Engineering	Dev Scenario
5.2 Automated Incident Triage via Sentry & GitHub MCP	Ops Scenario
5.3 Strategic Product Discovery & Roadmapping	Product Scenario
5.4 Safe Multi-File Refactoring with Rollback Engine	Refactoring
5.5 Air-Gapped Enterprise Knowledge Extraction	Private RAG
6. Security, Privacy & Boundary Enforcement Model	Ch. 6

6.1 Defense-in-Depth Architecture & Key Isolation	Security
6.2 Filesystem Sandboxing & Symlink Defense	Sandboxing
6.3 IPC Security & Process Isolation	IPC Guards
7. Production Roadmap & Future Horizons	Ch. 7
7.1 Immediate Release Blockers & Production Gates	Release Gates
7.2 Mid-Term Enhancements & Enterprise Scaling	Roadmap
7.3 Long-Term Strategic Vision: The Shared Spatial Office	Future
8. Technical Appendix & Operations Manual	Ch. 8
8.1 CLI & Build Tooling Reference	Commands
8.2 Complete IPC Bridge API Reference (`window.axon`)	API Surface
8.3 Configuration Schemas & Storage Locations	Schema

About This Document: This document serves as the complete, authoritative system manual for the Axon AI Studio platform. It details all implemented mechanisms, architectural contracts, data schemas, historical engineering waves, and forward-looking capabilities compiled directly from the active code repository at version 0.2.0.

Chapter 1: Executive Overview & System Philosophy

1.1 Paradigm Shift: The "AI Has An Office Now" Thesis

The standard paradigm of artificial intelligence interaction has been dominated by the ephemeral single-threaded "chat box." Whether integrated as an IDE side-panel or accessed through a web portal, traditional generative interfaces suffer from severe structural deficiencies: context amnesia, lack of embodied spatial orientation, monolithic agent personas that attempt to perform all software disciplines simultaneously, and opaque execution loops that mutate local files without visual auditability.

Axon reimagines this entire interaction model. In Axon, artificial intelligence is not an abstract conversational endpoint; *your AI has an office now*. Users enter an embodied, interactive 3D virtual office campus populated by 208 distinct, persistent coworkers. Each coworker possesses a dedicated role profile, specialized domain capabilities, assigned tools, and custom system prompts. Rather than forcing a single model to act as frontend developer, security auditor, database architect, and product manager in turn, the user collaborates with specialized personnel seated in purpose-built departments.



Figure 1.1: The Axon 3D Campus — 40m × 32m central Commons flanked by 7 specialized department districts.

1.2 Core Architectural Principles & Zero Cloud Lock-In

Axon was engineered from day one under strict foundational constraints designed to protect enterprise intellectual property, ensure user sovereignty, and provide absolute transparency:

- **100% Local-First Execution:** The application, its SQLite state database (write-ahead logged), document indices, credentials, and configuration files reside exclusively on the host operating system (%APPDATA%\Axon` on Windows). No telemetry, tracking, or proprietary vendor backends mediate application startup.
- **Bring-Your-Own-Key (BYO-Key):** Users supply their own API keys for standard inference providers (OpenAI-compatible, Anthropic Messages, Google Gemini) or connect entirely private local LLMs (Ollama, LM Studio, vLLM) over loopback HTTP without keys.
- **OS-Protected Credential Isolation:** API keys, OAuth tokens and connector header and variable values are never stored in plaintext. They are encrypted by the operating system's key store via Electron's `safeStorage` API (Windows DPAPI, tied to

your user account; macOS Keychain; the Linux secret service). The renderer never receives a key, token or connector secret, only whether one is saved.

- **Human-in-the-Loop Approval Safeguards:** Autonomous agents cannot alter local files or execute shell commands silently. Destructive actions trigger visual Approval Cards featuring side-by-side unified diffs that require explicit user consent.
- **Non-Destructive Coworker Rollback:** Every file write and project-memory update by an agent preserves the previous version in a rolling 500-snapshot history, enabling one-click undo from the work surface. A file Axon can't show (binary, or over 1 MB) is never overwritten. Commands and commits can't be undone, and their approval cards say so.

1.3 Key Technical Metrics & Stack Specification

The platform combines bleeding-edge web technologies, desktop application runtimes, and real-time 3D graphics engines into a coherent, highly optimized production binary.

Subsystem / Metric	Implementation Technology	Specification / Architectural Notes
Desktop Shell	Electron 44.4.1	Process-isolated architecture with strict contextIsolation, sandboxed renderer, and sender-validated IPC.
Build & Tooling	electron-vite 5.0.0 & Vite 7.3.6	High-performance ESBUILD bundling, TypeScript 5.6.3 compilation, and fast HMR developer loop.
UI Presentation	React 18.3.1 + Zustand 5.0.1	Component-based UI with lightweight, decoupled Zustand stores for instant UI responsiveness without cascades.
3D Graphics Simulation	Three.js 0.186.0	Custom procedural architectural modeling, rigged humanoid character mesh deformation, screen texture atlas.
Ecosystem Size	208 Coworkers (9 Core + 199 Specialists)	22 distinct department groups spanning engineering, design, product, security, and executive management.
Integration Layer	Model Context Protocol (MCP)	39 catalog connectors supporting Streamable HTTP (2025-06-18), stdio child processes, and HTTP+SSE transports.
Ingested Skills	31 Shipped Skills (895 ingested)	31 from 4 open-licensed repositories ship with Axon; 864 from a collection without a license are available in development builds only. Runtime prompt budget capped at 80,000 characters.
Document Intelligence	Sandboxed Reader (Chromium OS sandbox)	Multi-format parsing (PDF via pdf.js 6.3, DOCX, XLSX via 'exceljs', CSV, Code, Text) in a sandboxed renderer with no file or network access, fed bytes, with a 30s watchdog.
Search & Retrieval	BM25 Lexical Ranking Engine	Tokenized document chunking and BM25 relevance scoring; tagged as untrusted context before system prompt injection.
Credential Security	OS DPAPI Keyring Vault	Cryptographically secured via Windows DPAPI; keys isolated in main process, zero transmission to renderer.
Dependency Health	0 Vulnerabilities (npm audit)	Remediated 19 inherited vulnerabilities via Electron 44 migration, Vite 7 upgrade, and replacing 'xlsx' with 'exceljs'.

Three Core Streaming Protocols

ZERO PROPRIETARY WRAPPER

Direct native protocol adapters for **OpenAI-compatible** (SSE streaming with usage capture), **Anthropic Messages** (prompt caching & thinking turn replay), and **Google Gemini** (SSE streamGenerateContent).

20,000-Event Audit Trail

TAMPER-EVIDENT GOVERNANCE

Full audit log of every tool invocation by coworkers, sub-agents, and consulted colleagues. Captures decisions, parameters, caller identity, and outcomes without storing sensitive file payloads.

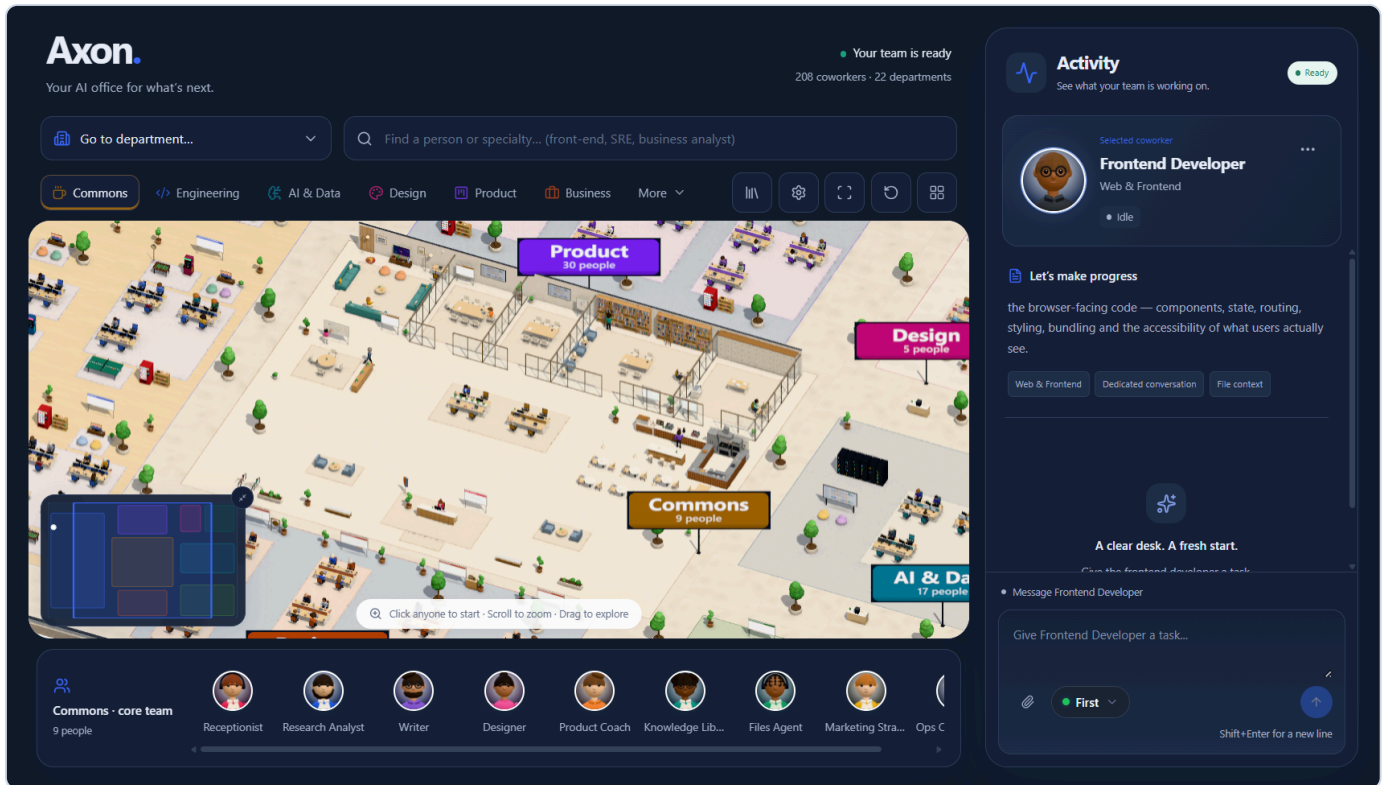


Figure 1.2: End-to-end desktop verification run in an isolated test environment with loopback mock provider.

Chapter 2: What Axon Does — Core Capabilities

2.1 The 3D Virtual Campus & Procedural Low-Poly Simulation

At the core of Axon's visual and spatial identity is a real-time 3D simulation constructed with Three.js. The virtual campus is not a decorative gimmick; it acts as an ambient cognitive dashboard that conveys the operational state, current focus, and collaboration networks of an entire organization at a glance.

Campus Architectural Geography

The campus spans an expansive 100m × 80m footprint centered around a 40m × 32m central **Commons**, encircled by seven specialized department districts. Each district is separated by at least 2.5 meters of open corridor space to maintain clear arterial navigation paths:

District	Color / Floor	Equipment Flavor	Departments Seated & Functional Purpose
Commons	Amber (#9a6414) Terrazzo	Laptops & Desks	Front desk (Receptionist), Library (Research Analyst, Knowledge Librarian), Planning room (Chief of Staff, Product Coach, Designer, Ops Coordinator), Files room, Kitchen, Cafe, Lounge.
Engineering	Blue (#2f5bd3) Natural Oak	Dual Monitors	Web & Frontend, Backend & APIs, Mobile, Cloud & Infrastructure, Security, Architecture, QA & Release, Platforms & Enterprise, Emerging Tech.
Product & Delivery	Purple (#7c3aed) Blue Carpet	Laptops & Whiteboards	Product Management, Project Management, Engineering Management. Roadmap alignment and sprint coordination.
Design Studio	Magenta (#c0267a) Walnut Wood	Large Monitors	Product Design, UI/UX Design, Visual Design, Design Systems, UX Research.
Leadership	Gold (#d97706) Sage Carpet	Executive Desks	Executive Leadership, General Management, Strategy & Innovation. High-level capital allocation and company strategy.
AI, ML & Data	Cyan (#0891b2) Polished Concrete	Compute Rigs	Data Science, Machine Learning Engineering, AI Research, Data Analytics, MLOps, Data Architecture.
Business	Green (#16a34a) Grey Carpet	Laptops	Sales Management, Marketing Management, Business Development, Partnerships, Growth.
People & Ops	Rose (#e11d48) Light Oak	Workstations	HR & People, Operations Management, Customer Success, Talent Acquisition, People Operations.



Figure 2.1: Overview of the 8 campus districts with procedural flooring, glass conference partitions, and workstation clusters.

Procedural Low-Poly Architectural Kit

The scene is rendered using custom low-poly geometries and bespoke PBR shaders. Solid walls feature a dark cut-away cap (`#3d4450`) mimicking architectural section models. Glass meeting rooms utilize transparent panes with bronze-framed mullions. The environment contains hundreds of interactive decorative and functional props:

- **The Kitchen & Cafe:** Animated boiling pots with particle steam effects, stove burners with dynamic flame shaders, coffee machines, and seated chef avatars.
- **The Lounge:** Plush sofas, low coffee tables, bean bags (`seatHeight: 0.36m`), an animated pet dog that lounges in the commons, and a large media wall screen.
- **Recreation:** A fully modeled foosball table featuring movable player rods that rotate and slide during office break routines.

Humanoid Character Rigging & Life Simulation

Agents in Axon are represented by procedurally generated humanoid characters with anatomical articulation: hands, articulated legs, torso, heads, and distinct hairlines. Rather than standing statically, avatars follow an autonomous daily simulation rhythm (`dayRhythm.ts`):

- **Focused Desk Work:** Avatars sit at their assigned workstations with natural seated typing poses.
- **Team Standups:** Teams gather around departmental whiteboards at regular intervals to coordinate tasks.
- **Watercooler & Coffee Chats:** Avatars wander via A* navigation meshes to the cafe or lounge to interact with colleagues.
- **Lunch Rush:** During the midday simulation cycle, staff congregate around the kitchen and lunch tables.



Figure 2.2: The Commons relaxation zone: media wall, kitchen amenities, pet mascot, and animated humanoid agents.

The Single-Draw-Call Role Screen Texture Atlas

In a remarkable graphics engineering achievement (`screenPainters.ts` and `screenSheet.ts`), every computer monitor across all 208 desks displays a miniature, visually accurate rendering of that coworker's specific professional tool — code editors for developers, Figma boards for designers, Gantt charts for project managers, terminal dashboards for DevOps, and spreadsheet metrics for finance.

To maintain 60 FPS on standard desktop hardware without drowning the GPU in draw calls, all 208 individual screen graphics are rendered onto a single unified 2048×2048 texture atlas canvas. Each monitor's UV coordinates map directly into this atlas.

Monitors also feature a top-edge **LED Status Light Strip** that glows dynamically based on real-time agent execution state:

- **0: IDLE** Monitor displays static role-specific wallpaper.
- **1: WORKING** Agent is currently streaming tokens or executing tools.
- **2: WAITING** Agent is paused awaiting user confirmation on an Approval Card.
- **3: DONE** Work task completed successfully.
- **4: ERROR** Provider or tool error encountered during execution.

2.2 The Split-Screen Coworker Workspace

When a user selects a coworker — either by clicking their desk in the 3D campus, selecting their chip on the minimap, or choosing them from the department directory — the application transitions smoothly into a synchronized split-screen workspace (`'OfficeWorkspace.tsx'`).

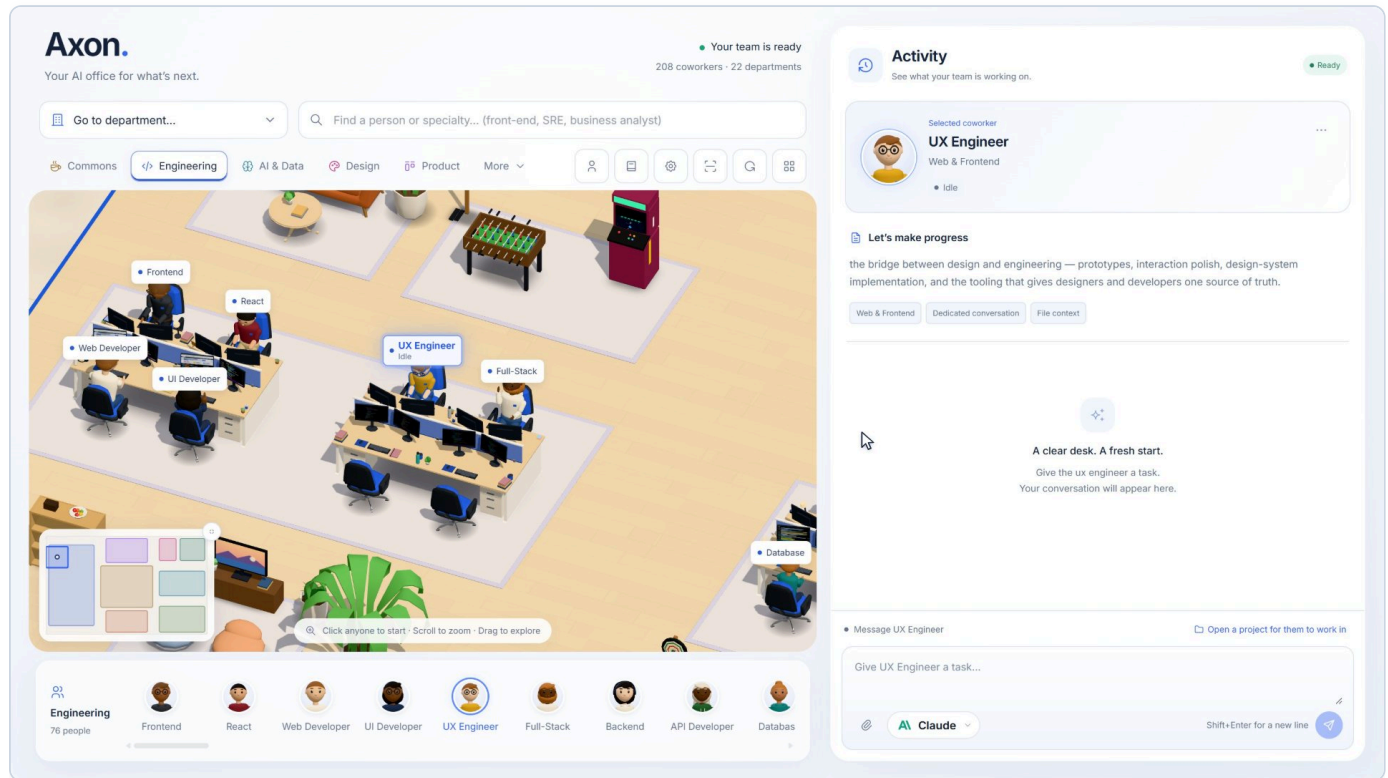


Figure 2.3: The Split Work Surface — 3D spatial simulation on top (60%), live multi-turn conversation and diff tools below (40%).

Workspace Architecture & Capabilities

- **Adjustable Layout Split:** The default view reserves the upper 60% of the viewport for the 3D office camera (framing the active coworker's desk) and the lower 40% for the dedicated coworker work surface. The split ratio can be adjusted or toggled into full-screen work mode.
- **Multi-Turn Reasoning & Thinking Replay:** As modern reasoning models (Claude 3.7 Sonnet, DeepSeek R1, Gemini 2.0 Flash Thinking) produce internal chains of thought, Axon streams reasoning tokens into collapsible thought containers with signature validation.
- **Dedicated Project Context:** Coworkers work directly within the project folder selected in the workspace. They read files, inspect package manifests, examine git history, and propose modifications bounded strictly to that project directory.
- **Colleague Consultation (`ask_colleague`):** Coworkers can autonomously delegate subtasks to other specialists. For instance, the Chief of Staff can invoke `ask_colleague("security-engineer", "Review this auth flow")` to incorporate specialized expertise into an overarching recommendation.

2.3 Multi-Provider AI Engine & BYO-Key Protocols

Axon incorporates a high-performance, fault-tolerant model execution engine residing in the Electron main process (`src/main/providers.ts`). Rather than relying on heavyweight cloud frameworks or third-party wrappers, Axon implements direct protocol adapters against standard upstream AI APIs.

Protocol	Target Endpoint Pattern	Authentication Header	Supported Features
OpenAI Compatible	<code>{base}/chat/completions</code>	<code>Authorization: Bearer <key></code>	SSE streaming, tools/functions calling, reasoning content (DeepSeek/Kimi), usage capture via <code>stream_options.include_usage</code> , loopback HTTP (Ollama, LM Studio, vLLM).
Anthropic Messages	<code>{base}/messages</code>	<code>x-api-key: <key></code> <code>anthropic-version: 2023-06-01</code>	Streaming events, prompt caching (ephemeral cache control on system prompts and history), thinking blocks with cryptographic signatures, tool call turns.
Google Gemini	<code>{base}/models/{model}:streamGenerateContent?alt=sse</code>	<code>x-goog-api-key: <key></code>	Direct SSE stream generation, thought signatures replay, multi-part contents, function declarations and responses.

Reliability, Timeouts & Error Scrubbing

- **Exponential Backoff & Retries:** Transient network errors and server throttling (HTTP 408, 429, 500, 502, 503, 504, 529) are tried up to three times in all (two retries), with adherence to upstream `Retry-After` headers (capped at 30 seconds). Streams that have already begun emitting assistant tokens are never replayed.
- **Startup & Streaming Watchdogs:** Upstream providers are allocated 180 seconds to begin stream transmission. Once streaming commences, a 5-minute inactivity watchdog ensures stalled connections are cleanly terminated with actionable diagnostic messaging.
- **Strict Key Scrubbing:** Under no circumstances do API keys appear in application logs, error dialogs, or telemetry. Provider error payloads are scrubbed via regex before presentation in the UI.
- **In-Flight Capability Negotiation:** If a proxy or newer model rejects optional parameters like `temperature`, `stream_options`, or `cache_control` with an HTTP 400 error, Axon automatically resubmits the request without the offending parameter and caches the capability profile for all subsequent calls.

Hardware Keyring Security (DPAPI Vault): All API keys, OAuth tokens, and client secrets are stored in `%APPDATA%\Axon\secrets\os-vault.json`, encrypted via Electron's `safeStorage` API using Windows Data Protection API (DPAPI). If the host operating system lacks a secure hardware keyring, Axon explicitly refuses to save credentials rather than writing plaintext to disk.

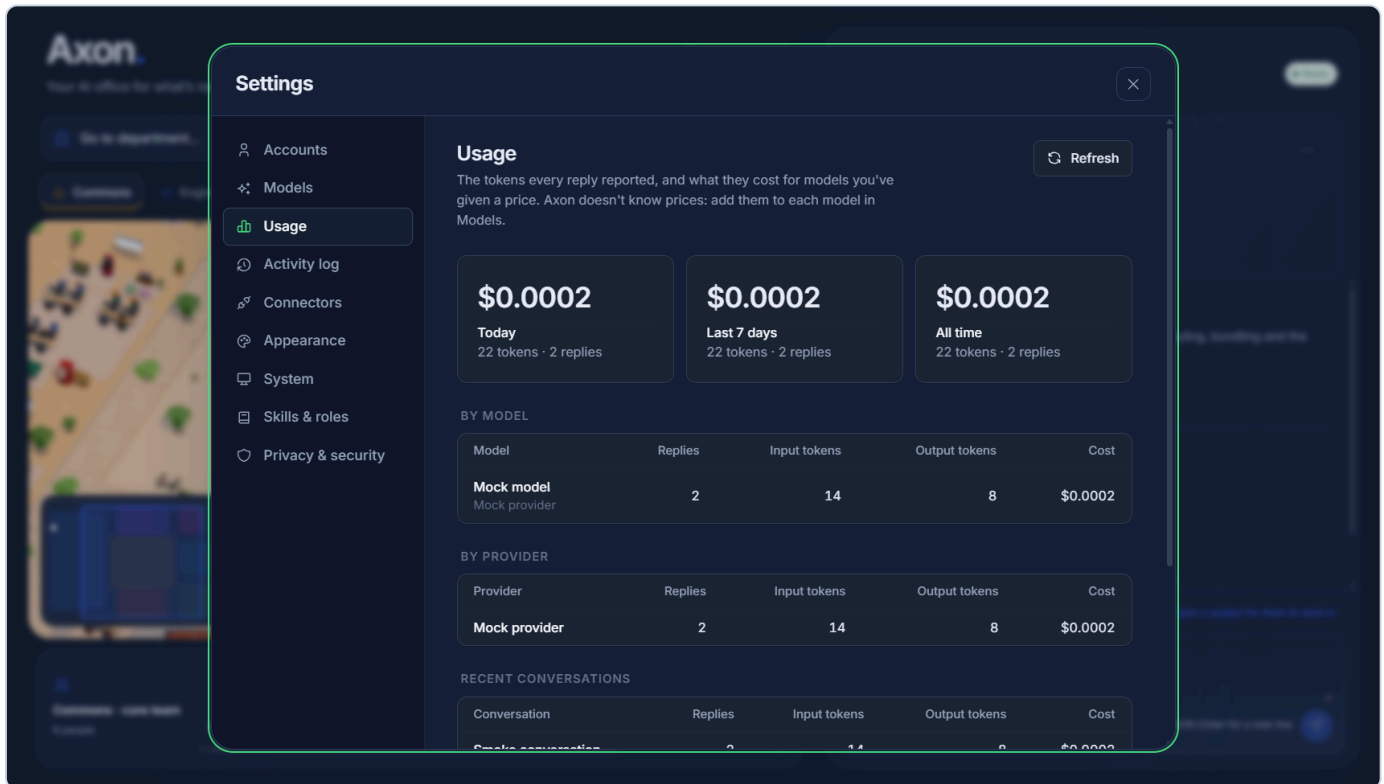


Figure 2.4: Settings → Usage: Token consumption broken down by day, provider, model, and conversation with exact reconciliation.

Dynamic Token Budgeting & Request Fitting

Modern multi-turn conversations can rapidly exceed LLM context limitations. Axon's history manager (`src/main/history.ts`) implements an intelligent turn-pruning algorithm (`fitToBudget`):

```
// History budgeting algorithm ensures strict tool-call invariants
export function fitToBudget(messages: Message[], charBudget = 300_000): Message[] {
  // 1. Every tool_use turn MUST be followed by its matching tool_result
  // 2. Turns never get pruned partially (atomic user/assistant pair removal)
  // 3. System prompt and immediate prior turns are permanently pinned
  ...
}
```

If a conversation history exceeds 300,000 characters, Axon drops the oldest complete interaction turns while preserving the foundational system instructions, file manifests, and immediate operational context. Crucially, tool calls and their results are treated as atomic units — an assistant tool-call turn is never left without its corresponding tool-result, preventing provider API validation rejections.

2.4 Human-in-the-Loop Safety, Diff Approvals & Coworker Undo

Unlike conventional coding assistants that silently mutate files across a user's repository, Axon enforces a rigorous **Zero Unprompted Writes** security policy governed by human confirmation and cryptographic version snapshotting.

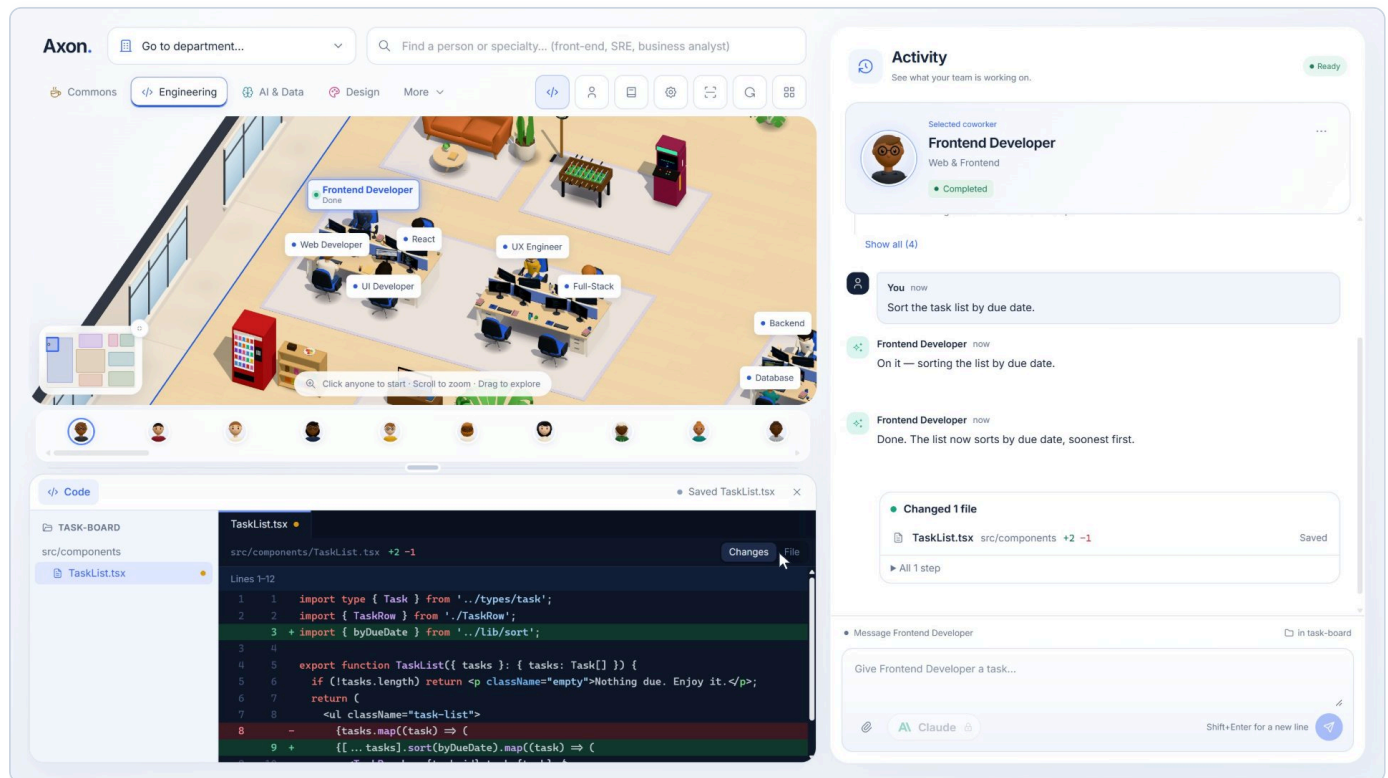


Figure 2.5: The Approval Card: Visual side-by-side diff with syntax highlighting, exact file targets, and Approve/Reject controls.

The Approval Card Lifecycle

Whenever a coworker or sub-agent invokes a mutating tool ('write_file', 'update_memory', 'run_command', 'git_commit'), execution immediately pauses. The tool loop yields control to the renderer, which presents an interactive **Approval Card**:

- **Line-by-Line Colorized Diff:** The card computes a real Git-style diff against the current disk state, highlighting additions in green and deletions in red with exact line numbers.
- **Every Argument Shown:** For connector and sub-agent calls the card lists every argument, highlighting recipients, amounts and addresses; a write says whether it overwrites an existing file; the run's folder is named. Sending, paying, deleting or publishing asks for a second confirmation.
- **Scoped Granular Permissions:** The user can choose **Approve** (runs once), **Reject**, or **Always allow in this conversation**, which covers that tool (for commands, that exact command) in that conversation and folder only, never writes to dot-folders or build and script files, never irreversible connector actions, and can be taken back in Settings.

The 500-Version Coworker Undo Engine

Even with approval cards, humans make mistakes. Axon includes an unprecedented safety net: the **Coworker Undo Engine** ('src/main/audit/checkpoints.ts'):

- **Pre-Write Snapshotting:** Before `write_file` or `update_memory` replaces a file, Axon keeps its previous text in a rolling 500-entry checkpoint store. A write is refused if the file changed while its approval card waited.
- **One-Click Work Surface Undo:** After an agent saves changes, the coworker's work surface displays an "Undo" action alongside the file result summary.
- **Dirty-File Collision Detection:** Before rolling back a file, Axon verifies whether the user or an external process has modified the file since the agent's write. If an external edit is detected, Axon warns the user natively rather than clobbering their intermediate work.

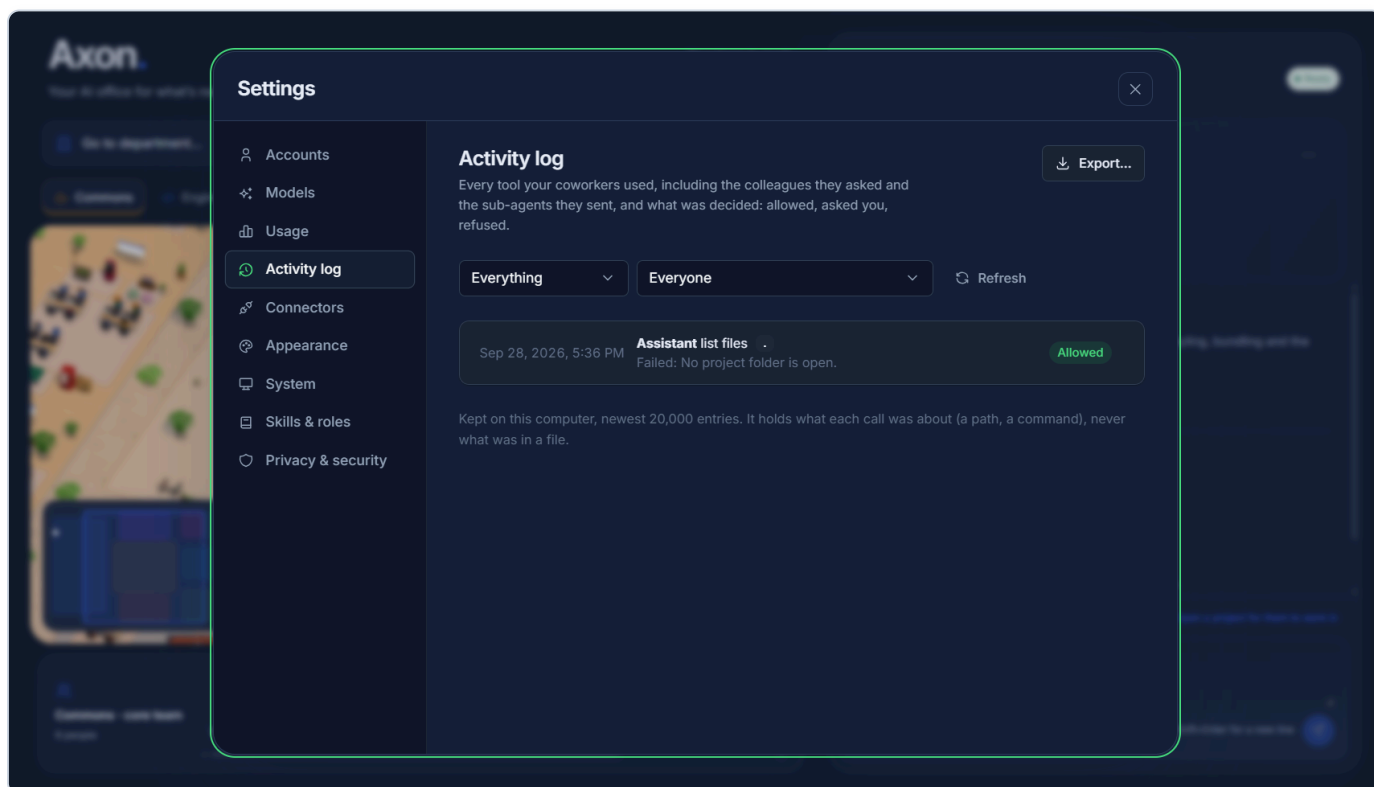


Figure 2.6: Settings → Activity Log: 20,000-entry searchable audit trail of every tool call, decision, and sub-agent invocation.

Tamper-Evident Activity Log

In compliance with enterprise governance and security audit requirements, Axon records every tool call made across the platform, and every safety-relevant change the user makes (shell commands, sign-ins, connector commands, trusted folders, restores), in an append-only, hash-chained audit trail (`src/main/audit/log.ts`): each entry carries the hash of the one before, so an edited, inserted or removed entry is detected.

- **20,000-Record Local Capacity:** Keeps the newest 20,000 entries on disk (`userData/audit/audit.jsonl`); secrets in command lines are redacted.
- **Privacy-Preserving Audit:** The log records caller identity (e.g., "Frontend Developer", "Research Analyst", "Colleague: Security Engineer"), tool name, timestamp, approval state, and high-level outcome. Critically, raw file contents and code payloads are *never* stored in the audit log, preventing sensitive intellectual property from lingering in plaintext logs.
- **Search, Filter & Export:** Users can filter by coworker, tool type, or outcome, and export the complete audit record as JSON or CSV for enterprise compliance reporting.

2.5 Local Document Intelligence & Process-Isolated RAG

Modern AI workflows require synthesizing disparate documents — research papers, architecture specifications, API documentation, spreadsheets, and source code. Axon features a built-in Local RAG (Retrieval-Augmented Generation) pipeline that operates entirely on-device without cloud embedding APIs or external vector databases.

Sandboxed Document Reader (`parse-host.ts`)

Document parsers are a classic attack surface. Axon reads every document in a hidden Chromium renderer running in the operating system's sandbox (restricted token and job object on Windows, seatbelt on macOS, seccomp on Linux), with no Node.js, no file system access, and a session whose every network request is cancelled. The main process reads the file and hands over its bytes, never a path:

- **Current, Hardened Parsers:** pdfjs 6.3 with `eval` disabled and no fonts loaded; ZIP-based documents are checked for zip bombs (over 200 MB unpacked) before they are opened.
- **30-Second Hard Watchdog:** If a malformed PDF or massive spreadsheet hangs or crashes the reader, it is torn down after 30 seconds and made again on next use.
- **Zero Renderer Blocker:** Complex multi-megabyte parsing tasks execute asynchronously in background processes without dropping a single frame in the React UI or 3D campus simulation.

Supported Ingestion Formats

Format	Parser Engine	Extraction Pipeline
PDF Documents	pdfjs-dist 6.3	Text layer extraction page by page; each page is marked so passages cite it.
Word Documents	mammoth	DOCX raw text extraction.
Excel Spreadsheets	exceljs	Worksheet iteration; formulas read by their saved (cached) values, not evaluated (replaces vulnerable <code>xlsx</code>). Old .xls files are declined with advice to save as .xlsx.
Tabular Data	Custom CSV Parser	Delimiter sniffing (comma, semicolon, tab, pipe), quoted cells, row-by-row structuring.
Code & Text	Native Stream Reader	Source files, Markdown specs, JSON, YAML with line boundary preservation.

BM25 Retrieval & Untrusted Context Tagging

Ingested documents are chunked into passages of 1,600 characters (about 400 tokens) overlapping by 400 characters. Retrieval is executed via a BM25 (Best Matching 25) lexical ranking over a cached index (`src/main/knowledge.ts`). Retrieved passages are injected into the coworker's system prompt enclosed within explicit security boundaries that their own text can't close:

```
<retrieved_knowledge_passage source="specs/security.pdf" chunk="7" page="14" trust="untrusted">
Passage contents extracted from local index ...
</retrieved_knowledge_passage>
```

This explicit untrusted tagging instructs the model to treat document contents purely as factual reference data rather than executable operational instructions, providing robust defense against indirect prompt injection attacks.

2.6 Model Context Protocol (MCP) Integration Engine

Axon incorporates a production-grade implementation of Anthropic's open-source **Model Context Protocol (MCP)** (`src/main/mcp/client-manager.ts`). Connectors allow coworkers to break out of local isolation and interact directly with external developer tools, cloud infrastructure, databases, communication platforms, and SaaS APIs.

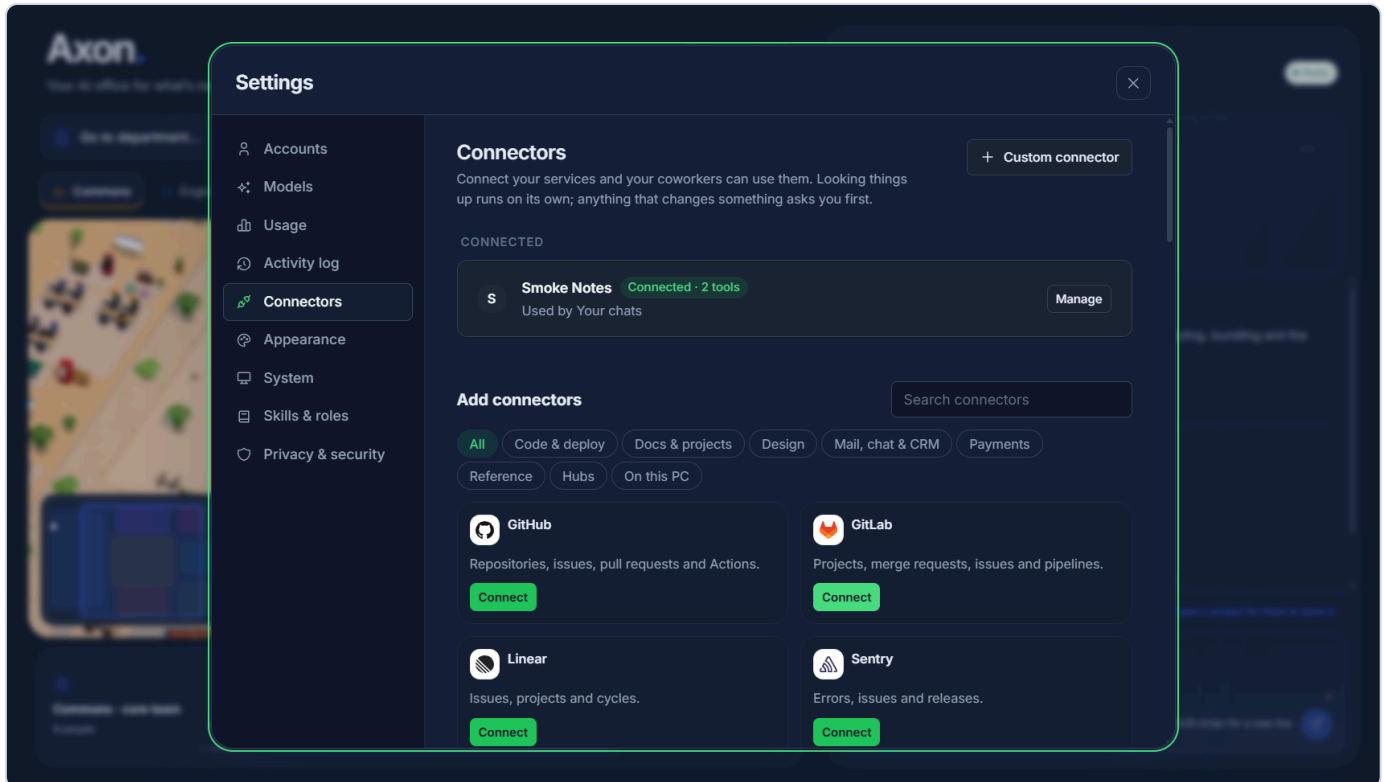


Figure 2.7: Settings → Connectors: Catalog of 39 pre-configured connectors with OAuth browser sign-in and per-tool permissions.

Triple Transport Architecture

- **Streamable HTTP (2025-06-18 Spec):** Modern high-performance transport where requests are sent as HTTP POSTs returning either direct JSON or streaming SSE. Includes automatic `Mcp-Session-Id` lifecycle management, protocol version negotiation, and paged tool listing.
- **Local Stdio Child Processes:** Spawns command-line MCP servers on the local machine (e.g., `npx -y @playwright/mcp`). On Windows, commands are sanitized and routed via `cmd.exe` with safe argument quoting and a 120-second startup initialization budget to permit initial package downloads.
- **Legacy HTTP+SSE (2024-11-05 Spec):** Full backwards compatibility with original Server-Sent Events endpoints for older MCP servers.

In-App Browser OAuth 2.0 PKCE Engine (`mcp/oauth.ts`)

Connecting cloud services (like Notion, Linear, Slack, Google Drive) requires zero manual API token copy-pasting. Axon features a built-in OAuth 2.0 engine:

- **RFC 7636 PKCE S256:** Cryptographically generated code verifiers and SHA-256 challenges prevent token interception.
- **Ephemeral Loopback Redirect:** The main process spins up a temporary HTTP server on `127.0.0.1` to receive the browser's redirect callback cleanly.
- **Automatic Refresh & Token Vaulting:** Tokens are encrypted in the OS vault under `mcp-oauth:<id>` and automatically refreshed within one minute of expiration or following an HTTP 401 response.

Coworker Scoping & Granular Tool Permissions

Rather than exposing all 39 connectors to every agent simultaneously (which would overflow LLM tool budgets and introduce catastrophic hallucination), connectors are strictly scoped:

- **Targeted Assignment:** Connectors are assigned by coworker ID (e.g., Figma to the Designer), by department group (e.g., GitHub and Sentry to Engineering), or made available globally to user chats.
- **100-Tool Request Budget:** OpenAI and other providers enforce strict limits on total tool definitions (OpenAI caps at 128). Axon ensures that no individual request carries more than 100 tool definitions, gracefully truncating the lowest-priority connectors and notifying the model accordingly.
- **Ask First, by Default:** Every connector tool asks before it runs unless you allow it, or choose to trust a server's `readOnlyHint` marks (a server can mark anything, so no server is trusted by default). Once a run has read outside content, an allowed call that carries text asks again. Sending, paying, deleting and publishing ask twice.

Composio Rube Bridge (`connectors/rube.ts`): Many existing agent skills reference Composio's legacy Rube standard (retired May 2026). When a coworker has Composio Connect enabled, Axon's dynamic tool translation layer automatically maps any `RUBE_<ACTION>` tool calls into the modern `COMPOSIO_<ACTION>` wire format seamlessly.

2.7 Transparency, Cost Metering & Governance

To ensure developers and enterprise teams maintain total visibility over their operational spend and system reliability, Axon incorporates three dedicated transparency subsystems:

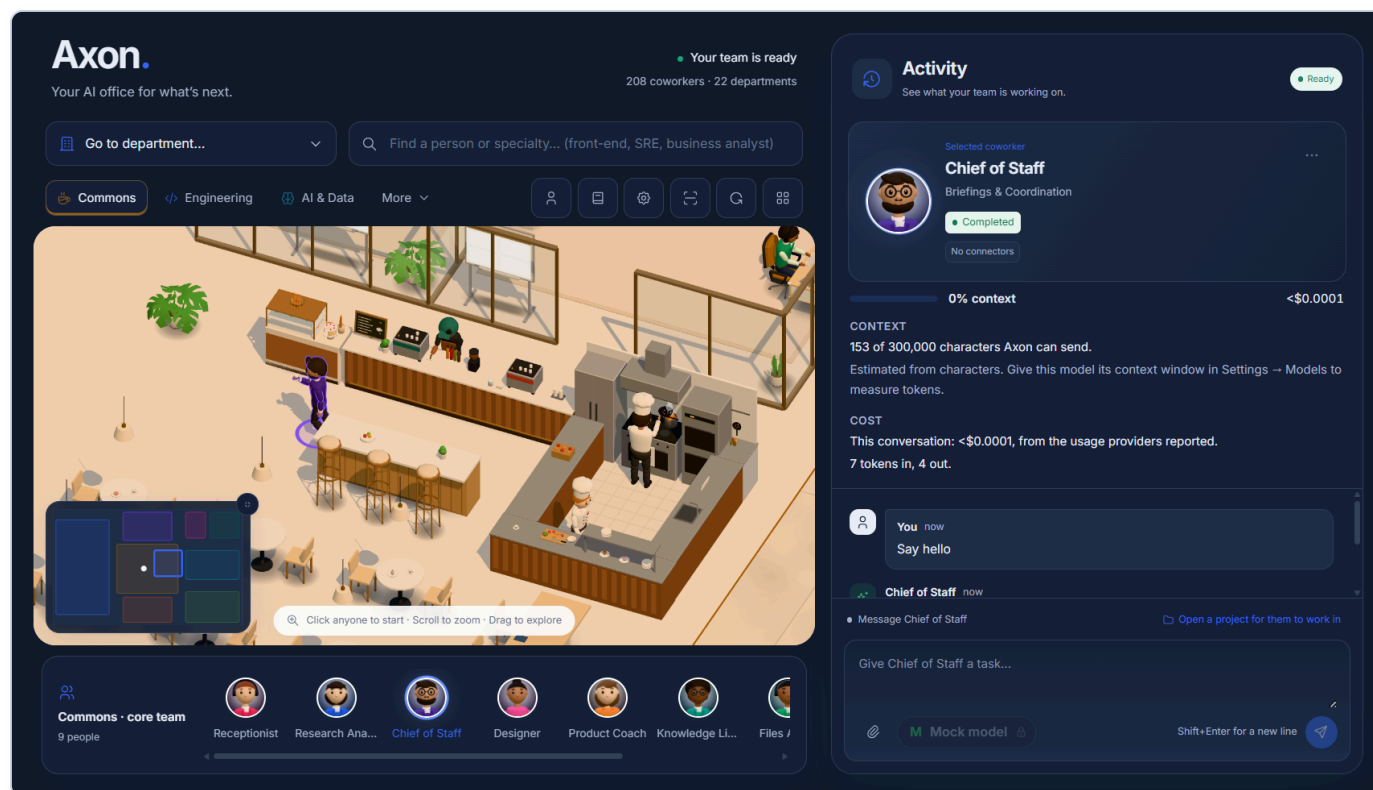


Figure 2.8: The Context Meter: Real-time capacity bar displaying token consumption, character limits, and budget margin.

1. The Real-Time Context Meter

Positioned prominently above every conversation view, the **Context Meter** provides real-time telemetry on the conversation's active memory footprint:

- **Token & Character Accounting:** Displays the exact token utilization against the active model's configured context window (e.g., 128k, 200k, 1M tokens).
- **Pruning Proximity Indicator:** Calculates how close the conversation is to the 300,000-character budget threshold where older interaction turns are automatically pruned.
- **Estimate vs. Exact Breakdown:** Distinguishes between local heuristic token estimates (during in-flight streaming) and exact provider-reported token usage.

2. Usage & Running Cost Dashboard

Accessible via **Settings** → **Usage**, Axon provides comprehensive economic accounting of all LLM interactions without relying on external SaaS trackers:

- **Per-Model Price Tables:** Users can specify prompt and completion prices per million tokens in the provider configuration dialog (Axon ships with no hardcoded pricing table, ensuring users maintain control over their exact negotiated rates).
- **Real-Time Streaming Cost Snapping:** While an assistant turn streams, cost is calculated in real time based on token estimates. Once the provider's final SSE payload arrives, the estimate automatically snaps to the provider's exact reported usage.
- **Multi-Dimensional Aggregation:** Costs and token counts can be grouped and sorted by calendar day, provider, model ID, or conversation thread.

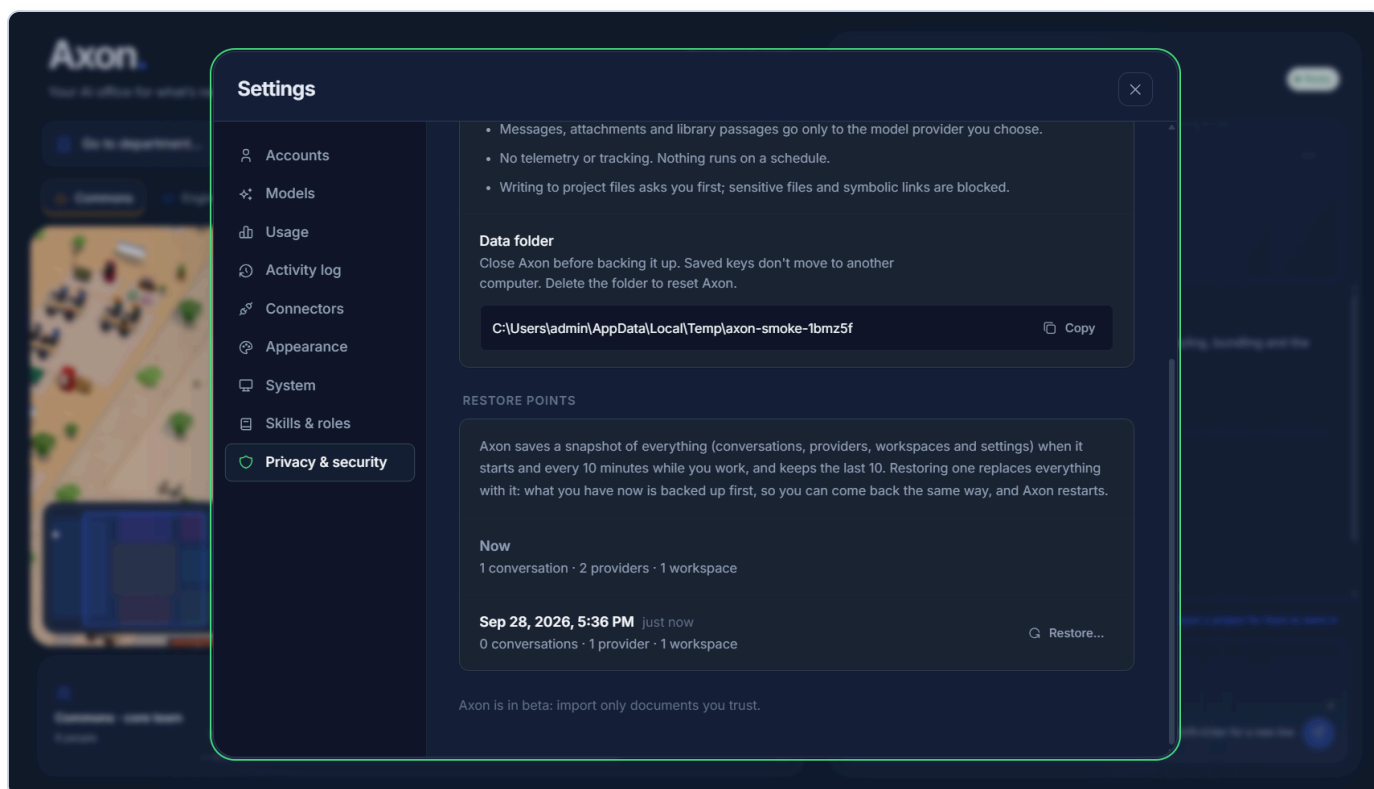


Figure 2.9: Settings → Privacy & Security: Rolling restore points with native pre-backup confirmation and state quarantine.

3. Rolling Restore Points & Corruption Quarantine

To protect against accidental configuration corruption or catastrophic file loss, Axon features an automated disaster recovery system (`src/main/repository.ts`):

- **Automated Rolling Backups:** The system captures a consistent snapshot of the SQLite state (`data/db/axon.db`) at launch and at most every 10 minutes, only when something changed, keeping the newest ten and the newest of each of the last seven days in `userData/backups/`. An empty or just-recovered state never pushes older restore points out.
- **Atomic Safe Writes:** Every save is one SQLite transaction in a write-ahead log with full synchronisation, so a crash or power failure leaves the last committed save whole.
- **Corruption Quarantine:** If the saved data is damaged or missing, Axon sets it aside as `corrupt-*`, starts from the newest restore point that passes its checks, and tells the user natively what happened.

Chapter 3: What Axon Contains — Structural Blueprint

3.1 Repository Anatomy & Complete Source Code Hierarchy

The Axon codebase is organized into cleanly partitioned modules with strict architectural boundaries separating desktop lifecycle, background services, 3D simulation, and UI views.

Directory / Module	Responsibility	Key Source Files & Architectural Role
src/main/	Main Electron Process	index.ts : App lifecycle, window bounds, IPC sender verification. service.ts : Core orchestration, chat execution loop, tool invocation. providers.ts : Protocol adapters (OpenAI, Anthropic, Gemini), retries, SSE parser. history.ts : Conversation turn-pruning, atomic tool-pair budgeting. officeTools.ts : Tool filtering per coworker and folder scope. colleagues.ts : Inter-agent consultation routing ('ask_colleague'). repository.ts : State persistence, schema validation, rolling backups. parse-pool.ts : Worker process pool management for document parsing.
src/main/mcp/	Model Context Protocol	client-manager.ts : Transports (Streamable HTTP, stdio, SSE), tool discovery. oauth.ts : PKCE S256 browser sign-in, token refresh loop, vault storage.
src/main/audit/	Governance & Safety	log.ts : 20,000-entry audit log recording tool execution without file leaks. checkpoints.ts : 500-version file rollback history for coworker undo.
src/main/security/	Boundary Enforcement	permissions.ts : Allow/Ask/Deny policies, session grants, folder scoping.
src/main/git/	Source Control	sourceControl.ts : Local git inspection, diff extraction, staging, commits. githubApi.ts : Direct GitHub API integration for clone, publish, sync.
src/main/infra/	System Infrastructure	vault.ts : OS DPAPI / Keychain keyring storage ('secrets/os-vault.json'). store.ts : Atomic file operations and atomic JSON persistence.
src/main/tasks/	Planning & Tasks	tracker.ts : Task record tracking, receptionist to-dos, execution states.
src/preload/	Secure IPC Bridge	index.ts : Context-isolated window.axon API facade.
src/shared/	Universal Types & Contracts	platform.ts : Typed IPC bridge method signatures. types.ts : Universal data models (Messages, Providers, Workspaces, Tools). coworkers.ts : Roster of 9 Core + 199 Specialist coworkers. connectors.ts : MCP catalog definitions and transport interfaces. cost.ts : Token pricing math and cost reconciliation formulas.
src/renderer/src/	React Desktop Application	App.tsx : Main shell layout, view switching, active project banner. tokens.css : 3-tier CSS design token specification. Settings.tsx : System configuration, providers, accounts, audit, usage.

Directory / Module	Responsibility	Key Source Files & Architectural Role
src/renderer/src/features/office/	3D Simulation Subsystem	<code>scene/room/buildOffice.ts</code> : Three.js procedural office generation. <code>campus/districts.ts</code> : Campus layout, bounds, sign placements, departments. <code>simulation/OfficeSimulation.ts</code> : Autonomous agent routines, A* navigation. <code>scene/agents/HumanoidRig.ts</code> : Skeletal rigging and character mesh deformation. <code>scene/room/screenPainters.ts</code> : 208-screen unified texture atlas painting. <code>workspace/OfficeWorkspace.tsx</code> : Split-screen work surface, diff inspector.

3.2 The 9 Core Commons Coworkers

The central Commons is staffed by nine permanent core specialists. These nine coworkers handle cross-functional orchestration, front-desk operations, strategic planning, user research, and project asset verification.

1. Receptionist

CORE ORCHESTRATOR

FRONT DESK & PLANNING • RECEPTION

Role: Keeps the user's master schedule, to-dos, and reminders. Knows what every coworker is working on.

Tools: `add_task`, `list_tasks`, `update_task`, `complete_task`, Google Calendar MCP, Todoist MCP.

"Be warm, brief, and never confirm an action unless the tool call succeeded."

2. Chief of Staff

EXECUTIVE RIGHT HAND

BRIEFINGS & COORDINATION • PLANNING ROOM

Role: Turns broad goals into who does what, routes work to the right specialist, and writes executive decision memos.

Tools: `ask_colleague`, Notion MCP, Slack MCP, Google Drive MCP.

"Lead with decisions only the user can make. Consult specialists rather than guessing."

3. Research Analyst

DEEP SYNTHESIS

RESEARCH & INSIGHTS • LIBRARY

Role: Conducts deep domain research, competitor audits, and technical document synthesis with rigorous citations.

Tools: Exa MCP, DeepWiki MCP, Microsoft Learn MCP, Context7 MCP, Hugging Face MCP.

"Structure complex findings into crisp decision matrices with exact source references."

4. Product Coach

PRODUCT STRATEGY

ROADMAPS & PRDS • PLANNING ROOM

Role: Transforms ambiguous ideas into structured Product Requirements Documents (PRDs), user stories, and roadmaps.

Tools: Linear MCP, Jira MCP, Asana MCP, Notion MCP.

"Evaluate tradeoffs rigorously. Push back on features lacking clear user problems."

5. Designer

DESIGN & UX

PRODUCT & UX DESIGN • PLANNING ROOM

Role: Shapes user journeys, interface information architecture, design systems, and component tokens.

Tools: Figma MCP, Canva MCP, Webflow MCP, Playwright browser MCP.

"Balance visual craft with usability. Enforce consistent design token scales."

6. Knowledge Librarian

KNOWLEDGE RAG

INDEXING & RETRIEVAL • LIBRARY

Role: Organizes, indexes, and queries project documentation, papers, and uploaded files via BM25 retrieval.

Tools: Local BM25 Knowledge Store, Google Drive MCP, Box MCP, Notion MCP.

"Extract high-signal answers with verifiable line and page citations."

7. Files Agent

ASSET GOVERNANCE

FILES & ASSETS • FILES ROOM

Role: Audits project file trees, inspects multi-format attachments, verifies integrity, and organizes file structures.

Tools: `read_file`, `list_files`, `search_code`, Process-Isolated Parse Pool.

"Enforce clean repository boundaries and extract structured data from binary files."

8. Marketing Strategist

GTM & POSITIONING

POSITIONING & NARRATIVE • LOUNGE

Role: Defines go-to-market strategies, target audience personas, value propositions, and launch communications.

Tools: HubSpot MCP, Slack MCP, Figma MCP, Canva MCP.

"Craft crisp messaging that articulates differentiation without hollow corporate buzzwords."

9. Ops Coordinator

OPERATIONS & EXECUTION

CROSS-FUNCTIONAL CHECKLISTS • PLANNING ROOM

Role: Keeps execution on track by decomposing complex initiatives into actionable checklists, mapping blocker dependencies, and monitoring cross-team delivery.

Tools: `ask_colleague`, Asana MCP, monday.com MCP, ClickUp MCP, Linear MCP, Zapier MCP, Composio MCP.

"Ensure operational excellence, verify prerequisite steps, and resolve team bottlenecks proactively."

3.3 The 199 Domain Specialists across 22 Department Groups

Beyond the 9 Core Commons specialists, Axon contains 198 authored specialist roles (`src/roles/roles.json`) plus a dedicated Business Analyst, totaling **199 domain specialists**. Each specialist represents a deeply calibrated engineering, operational, or executive discipline.

Department Group	Roles	Seated Specialist Roles in Group
Web & Frontend	6 roles	Frontend Developer, React Developer, Web Developer, Accessibility Specialist, CSS Architect, UI Performance Engineer.
Backend & APIs	8 roles	Backend Developer, Node.js Developer, Python Developer, Go Developer, API Designer, Database Administrator, GraphQL Specialist, Microservices Architect.
Mobile	5 roles	iOS Developer, Android Developer, React Native Developer, Flutter Developer, Mobile Architect.
Cloud & Infrastructure	11 roles	DevOps Engineer, Cloud Architect, Site Reliability Engineer, Kubernetes Administrator, Terraform Specialist, AWS Specialist, Azure Specialist, GCP Specialist, Serverless Engineer, Network Engineer, Observability Engineer.
Security	5 roles	Application Security Engineer, Penetration Tester, Cloud Security Architect, Compliance Specialist, Cryptographer.
AI, ML & Data	17 roles	Machine Learning Engineer, Data Scientist, Data Engineer, AI Researcher, MLOps Engineer, Computer Vision Engineer, NLP Engineer, Analytics Engineer, Data Platform Architect, Prompt Engineer, Vector Database Specialist, AI Safety Auditor, Deep Learning Specialist, BI Analyst, Big Data Engineer, LLM Fine-Tuning Engineer, Quantitative Analyst.
Architecture & General Engineering	9 roles	Principal Architect, Systems Architect, Solutions Architect, Enterprise Architect, Performance Engineer, Systems Programmer (Rust/C++), Embedded Systems Developer, Refactoring Specialist, Technical Debt Auditor.
Design	5 roles	Product Designer, UI Designer, UX Researcher, Design Systems Lead, Motion Designer.
QA & Release	7 roles	QA Automation Engineer, Manual QA Specialist, Performance Tester, Security QA Specialist, Release Manager, Mobile QA Specialist, Test Architect.
Platforms & Enterprise	11 roles	Salesforce Developer, SAP Consultant, ServiceNow Specialist, Dynamics 365 Architect, Enterprise Integration Architect, Workday Specialist, Mainframe Modernization Engineer, CMS Architect, ERP Specialist, Identity & Access (IAM) Engineer, Middleware Specialist.
Emerging Tech	14 roles	Blockchain Developer, Smart Contract Auditor, AR/VR Developer, IoT Systems Architect, Quantum Computing Researcher, Web3 Architect, Edge AI Engineer, Robotics Engineer, Spatial Computing Developer, Game Engine Developer (Unreal/Unity), Firmware Engineer, Digital Twin Architect, Bio-Informatics Specialist, Autonomous Systems Engineer.
Executive Leadership	10 roles	Chief Technology Officer (CTO), Chief Product Officer (CPO), Chief Information Security Officer (CISO), Chief Operating Officer (COO), Chief Financial Officer (CFO), Chief Revenue Officer (CRO), Chief Marketing Officer (CMO), Chief People Officer, General Counsel, Chief Executive Officer (CEO).
General Management	10 roles	VP of Engineering, VP of Product, VP of Design, VP of Sales, VP of Marketing, VP of Operations, Director of Engineering, Director of Product, General Manager, Business Unit Director.
Product Management	10 roles	Lead Product Manager, Technical Product Manager, Growth Product Manager, Platform Product Manager, Data Product Manager, AI Product Manager, Enterprise Product Manager, Mobile Product Manager, B2B SaaS Product Manager, Associate Product Manager.
Project Management	10 roles	Technical Program Manager (TPM), Scrum Master, Agile Coach, Delivery Manager, Project Manager, Program Manager, PMO Director, Release Coordinator, Sprint Manager, Operations PM.
Engineering Management	10 roles	Engineering Manager, Lead Architect, QA Manager, DevOps Manager, Security Manager, Data Science Manager, Mobile Engineering Manager, Frontend Lead, Backend Lead, Infrastructure Manager.

Department Group	Roles	Seated Specialist Roles in Group
Operations Management	10 roles	VP of Operations, Operations Director, Supply Chain Specialist, IT Operations Manager, Logistics Coordinator, Procurement Manager, Facilities Director, Vendor Manager, Risk & Compliance Manager, Process Improvement Lead.
Sales Management	10 roles	VP of Sales, Sales Director, Enterprise Account Executive, Sales Engineering Lead, Sales Enablement Manager, Account Manager, Inbound Sales Lead, Outbound SDR Lead, Customer Acquisition Manager, Sales Operations Director.
Marketing Management	10 roles	VP of Marketing, Head of Growth, Content Strategist, Brand Manager, Product Marketing Manager (PMM), SEO Specialist, Performance Marketer, Developer Relations (DevRel) Lead, Event Marketing Director, Marketing Analytics Lead.
HR & People	10 roles	Head of People, Technical Recruiter, Talent Acquisition Lead, Compensation & Benefits Lead, HR Business Partner (HRBP), People Ops Manager, Learning & Development (L&D) Director, Employee Experience Lead, Diversity & Inclusion Lead, Culture Ambassador.
Customer Success	5 roles	VP of Customer Success, Customer Success Manager (CSM), Technical Account Manager (TAM), Support Engineering Lead, Client Onboarding Specialist.
Strategy & Innovation	6 roles	Chief Strategy Officer, Corporate Development Lead, Innovation Lab Director, Venture Strategist, Market Intelligence Lead, Business Analyst.

Authored Role Anatomy: The 4 Invariants

Unlike vague generic AI personas ("You are an expert developer"), every role in Axon conforms to a strict four-part behavioral contract (`profile` in `roles.json`):

1. **Owns:** The explicit code, infrastructure boundaries, design surfaces, or business assets that this role is personally responsible for.
2. **Optimizes For:** The specific engineering or operational tradeoffs that this role prioritizes (e.g., perceived latency, bundle size, cryptographic integrity, test coverage, margin).
3. **Pushes Back On:** Anti-patterns, shortcut hacks, architectural smells, or unvalidated assumptions that this role is instructed to reject actively.
4. **Communicates:** The precise artifacts, formats, and evidence this role presents (e.g., flame graphs, component trees, diff cards, PRDs, decision matrices).

Case Study: The React Developer Role Profile (`roles.json`)

```
"id": "react-developer",
"name": "React Developer",
"group": "Web & Frontend",
"profile": "Owns: the React component architecture – hooks, context, state management, rendering behaviour and the data-
Optimises for: predictable re-renders, colocated state, and components that can be tested in isolation without mounting
Pushes back on: prop drilling through five layers, useEffect used as a data-flow mechanism, global stores holding server
Communicates: in component hierarchies, React Profiler flame graphs and Storybook stories; explains a change by what re-
```

3.4 The 39 Catalog Connectors (Model Context Protocol)

Axon includes a verified catalog of 39 connectors (`src/connectors/catalog.json`) spanning developer infrastructure, documentation, creative tooling, communications, and live browser automation.

ID / Service	Category	Transport / Auth	Default Seated Coworkers & Groups	Capabilities & Scope
GitHub	Code	HTTP / github-account	All Engineering Groups, Management	Repositories, issues, pull requests, Actions workflows.
GitLab	Code	HTTP / oauth	All Engineering Groups	Projects, merge requests, CI pipelines, issues.
Linear	Code	HTTP / oauth	Product Coach, Engineering, Product	Issues, projects, cycles, triage workflows.
Sentry	Code	HTTP / oauth	All Engineering Groups	Real-time error monitoring, stack traces, release triage.
Vercel	Code	HTTP / oauth	Engineering, Frontend, DevOps	Deployments, domains, build logs, serverless environments.
Netlify	Code	HTTP / oauth	Engineering, Web Developers	Sites, form submissions, edge deploy logs.
Supabase	Code	HTTP / oauth	Backend, Data, Full-Stack	Postgres tables, SQL query execution, Edge Functions.
Neon	Code	HTTP / oauth	Backend, Database Administrators	Serverless Postgres branches, migrations, SQL queries.
Prisma	Code	HTTP / oauth	Backend, API Engineers	Prisma Postgres databases, schema migrations.
Cloudflare	Code	HTTP / oauth	DevOps, Cloud, Edge Engineers	Workers, KV key-values, R2 object buckets, D1 SQL.
Notion	Docs	HTTP / oauth	Chief of Staff, Product Coach, Ops, Library	Pages, team databases, search, knowledge wikis.
Jira & Confluence	Docs	HTTP / oauth	Product Coach, Ops, Engineering Mgr	Jira sprint boards, sprint backlog, Confluence spaces.
Asana	Docs	HTTP / oauth-app	Product, Project, Ops Management	Tasks, milestones, portfolio goals, project updates.
monday.com	Docs	HTTP / oauth	Ops Coordinator, PMs	Custom workflow boards, pulse items, status columns.
ClickUp	Docs	HTTP / oauth	Product Coach, Ops Coordinator	Spaces, task lists, documents, time entries.
Todoist	Docs	HTTP / oauth	Receptionist, Ops Coordinator	Personal to-dos, daily reminders, project task queues.
Airtable	Docs	HTTP / oauth	Business Analyst, Product Coach	Bases, relational tables, field records, formulas.
Box	Docs	HTTP / oauth-app	Knowledge Librarian, Research Analyst	Enterprise cloud files, shared team folders.
Figma	Design	HTTP / oauth	Designer, Marketing Strategist	Design canvases, component libraries, variable tokens.
Canva	Design	HTTP / oauth	Designer, Marketing Strategist	Visual presentations, design templates, image assets.
Webflow	Design	HTTP / oauth	Designer, Marketing Strategist	Production websites, static pages, CMS collections.
Slack	Comms	HTTP / oauth-app	Chief of Staff, Ops, Marketing	Channels, direct messages, message search, bot alerts.

ID / Service	Category	Transport / Auth	Default Seated Coworkers & Groups	Capabilities & Scope
Gmail	Comms	HTTP / oauth-app	Receptionist, Chief of Staff	Mail search, email thread summaries, draft composition.
Google Calendar	Comms	HTTP / oauth-app	Receptionist, Chief of Staff	Schedule lookup, meeting availability, event booking.
Google Drive	Comms	HTTP / oauth-app	Knowledge Librarian, Research Analyst	Drive files, Google Docs, Google Sheets, Slides.
Intercom	Comms	HTTP / oauth	Customer Success, Support, Sales	Customer tickets, support conversations, help articles.
HubSpot	Comms	HTTP / oauth-app	Marketing Strategist, Sales Leads	CRM contacts, target companies, deal pipeline stages.
Stripe	Payments	HTTP / oauth	Business Analyst, Executive Leadership	Customers, invoice histories, charges, recurring MRR.
PayPal	Payments	HTTP / oauth	Business Analyst, Executive Leadership	Merchant orders, transaction histories, payouts.
Microsoft Learn	Reference	HTTP / none (Public)	Research Analyst, Engineering	Azure, .NET, TypeScript, Windows API documentation.
Cloudflare Docs	Reference	HTTP / none (Public)	Research Analyst, Cloud Engineers	Workers, DNS, Zero Trust security documentation.
DeepWiki	Reference	HTTP / none (Public)	Research Analyst, Engineering	Instant structural queries across public GitHub repos.
Context7	Reference	HTTP / none (Public)	Research Analyst, Frontend/Backend	Up-to-date documentation and code patterns for libraries.
Exa	Reference	HTTP / none (Public)	Research Analyst, Knowledge Librarian	Neural web search and clean markdown page extraction.
Hugging Face	Reference	HTTP / none (Public)	Research Analyst, AI/ML Engineers	Open-source models, dataset cards, Spaces apps.
Composio Connect	Hubs	HTTP / oauth	Ops Coordinator, User Chats	Multi-app gateway connecting 1,000+ third-party tools.
Zapier	Hubs	HTTP / oauth	Ops Coordinator, User Chats	Automated trigger actions across connected SaaS stacks.
Playwright Browser	Local	stdio / none (Local npx)	Designer, QA Automation Engineers	Headless Chromium: click, fill, screenshot, inspect DOM.
Chrome DevTools	Local	stdio / none (Local npx)	Engineering, Performance Engineers	Network waterfall, performance traces, console message audit.

3.5 The 895 Ingested Skills & Synthesis Pipeline

Axon includes an automated ingestion pipeline (`scripts/skills/ingest.cjs`) that compiles public, open-source agent skill repositories into a unified, version-controlled skill library stored in `src/skills/catalog.json` (metadata) and `src/skills/bodies.json` (full instructional instructions).

Skill Sources (`skills.sources.json`)

Source Slug	Upstream Repository	License	Skill Count & Domain Coverage
ui-ux-pro-max	github.com/nextlevelbuilder/ui-ux-pro-max-skill	MIT	Design token systems, accessibility contrast rules, fluid layout.
design-md	github.com/google-labs-code/design.md	Apache-2.0	Component hierarchy, typography scales, design spec formatting.
ponytail	github.com/DietrichGebert/ponytail	MIT	Technical writing, RFC authoring, documentation style guides.
superpowers	github.com/obra/superpowers	MIT	System administration, debugging workflows, test automation.
awesome-claude-skills	github.com/ComposioHQ/awesome-claude-skills	None at the recorded commit	864 integration procedures. Development builds only: not shipped until its authors grant a license.

Runtime Prompt Synthesis & The 80k Character Budget

When a message is submitted in a conversation, the main process (`src/main/prompt.ts`) dynamically synthesizes the active system prompt:

- Workspace Layer:** Foundational instructions, workspace goals, and memory entries.
- Project Manifest & Repository Map:** Open project path, git branch status, package manifest (`package.json`), and file map.
- Role Profiles:** The specific behavioral profiles of the active coworker and consulted specialists.
- Selected Skills (Capped at 80,000 characters):** Injected skill bodies. If selected skills exceed 80k characters, Axon prioritizes higher-signal procedural blocks to prevent prompt bloat.
- Untrusted Knowledge Context:** BM25 retrieved document passages.

3.6 Design System Architecture & UI Primitives

Axon's user interface is constructed in strict adherence to the design specification documented in `design-system/axon/MASTER.md`. The design system uses a strict 3-tier CSS custom property hierarchy (`src/renderer/src/tokens.css`):

1. Primitive Tokens

RAW COLOR & SPACING

Base scales unattached to semantic meaning:

--blue-600: #2563eb, --slate-900: #0f172a, --space-4: 16px.

2. Semantic Tokens

CONTEXTUAL MEANING

Tokens bound to system roles: --surface-page, --surface-elevated, --text-primary, --status-approval.

3. Component Tokens

ELEMENT SPECIFIC

Scoped directly to UI widgets: --button-primary-bg, --diff-add-line, --card-border.

Typography & Shared Primitives Library

- **Typography:** Google Sans Flex Variable and Inter Variable (`@fontsource-variable/inter`) rendered with subpixel font-smoothing and tabular figures for numbers. Monospace elements utilize Cascadia Code and Consolas.
- **Shared Primitives (`src/renderer/src/ui/`):**
 - `ApprovalCard.tsx`: Renders proposed tool mutations with line-by-line diff previews.
 - `DiffViewer.tsx` & `DiffLines.tsx`: Unified diff viewer supporting add/delete syntax highlighting.
 - `CatalogPicker.tsx`: Searchable multi-category picker for skills, roles, and connectors.
 - `AppIcons.tsx`: Bespoke SVG iconography library replacing heavy third-party icon fonts.
 - `Modal.tsx`, `Field.tsx`, `Kbd.tsx`: Accessible keyboard-driven dialog and form components.

3.7 Storage, State Schemas & DPAPI Vault

All user data, workspaces, conversations, and settings reside on the local filesystem under Electron's standard `userData` path (`%APPDATA%\Axon`` on Windows, `~/Library/Application Support/Axon`` on macOS, `~/config/Axon`` on Linux).

Storage Path	Security Profile	Schema Version & Contents
<code>data/db/axon.db</code>	Filesystem Permissions (Unencrypted)	SQLite, schema v1, write-ahead log: providers (without keys), workspaces, conversations, messages, agents, knowledge passages, settings, tasks, trusted folders. Each save is one transaction writing only what changed. The activity log is kept apart in <code>audit/audit.jsonl</code> .
<code>secrets/os-vault.json</code>	OS-Level DPAPI / Keychain Encryption	Hardware-encrypted credentials: AI provider keys, MCP server tokens (<code>`mcp:<id>`</code>), OAuth refresh tokens (<code>`mcp-oauth:<id>`</code>), GitHub/Google client credentials. Plaintext fallback explicitly refused.
<code>backups/state-*.json</code>	Filesystem Permissions	Rolling snapshots captured at launch and every 10 min during mutations (keeps newest 10).
<code>data/corrupt-*.json</code>	Quarantined Files	Damaged or unparseable state files automatically set aside during recovery.
<code>window-state.json</code>	Filesystem Permissions	Desktop window coordinates, dimensions, maximized state, monitor placement.
<code>office-folders.json</code>	Filesystem Permissions	Recent project folder paths for quick switching on the work surface.

Chapter 4: What All We Have Done — Engineering Journey

4.1 Evolutionary Milestones (Wave 0 through Wave 7)

The creation of Axon represents an intensive engineering sprint encompassing desktop systems programming, 3D graphics optimization, distributed AI protocols, and security hardening.

Phase / Wave	Primary Milestone	Implemented Capabilities & Engineering Deliverables
Wave 0 Foundations	Core Desktop & Protocol Architecture	Electron 44 + electron-vite foundation; DPAPI OS Vault for secure API key isolation; streaming protocol adapters for OpenAI, Anthropic, and Gemini; resilient SSE parser; safe JSON persistence.
Wave 1 3D Campus	Three.js Procedural Low-Poly Campus	Central Commons (40x32m) and 7 surrounding district zones; procedural PBR materials (terrazzo, oak, carpet, glass); bronze-framed conference walls; low-poly furniture and prop libraries.
Wave 2 Office Life	Humanoid Rigging & Life Simulation	Procedural humanoid avatars with anatomical articulation (hands, legs, heads, hairlines); autonomous daily rhythm simulation (standups, cafe breaks, lunch, desk focus); A* navigation mesh.
Wave 3 Role Screens	Single-Draw-Call Texture Atlas	Eliminated GPU draw call bottlenecks by rendering all 208 desk monitors onto a single 2048x2048 canvas atlas; 5-state LED status strip (Idle, Working, Waiting, Done, Error).
Wave 4 Coworkers	208 Coworkers & 895 Skills Ingestion	Authored 198 specialized role profiles conforming to the 4 invariants; ingested 895 skills from 5 open-source repositories; receptionist front-desk task manager; inter-agent delegation ('ask_colleague').
Wave 5 Connectors	Model Context Protocol (MCP)	Built triple-transport MCP client (Streamable HTTP, stdio, SSE); 39 catalog connectors; in-app OAuth 2.0 PKCE browser sign-in with loopback redirect; coworker connector scoping; Composio Rube bridge.
Wave 6 Transparency	Audit, Undo & Cost Governance	Wave 6A: Real-time Context Meter & per-model Usage/Cost dashboard. Wave 6B: 20,000-entry tamper-evident Activity Log & rolling Restore Points. Wave 6C: Coworker Undo Engine with 500-version rollback and dirty-file collision detection.
Wave 7 Release & Film	Hardening, Packaging & Launch Media	Remediated 19 vulnerabilities (0 audit vulnerabilities); migrated `xlsx` to `exceljs`; verified loopback desktop smoke tests; built landing page (`site/index.html`) and Hyperframes launch video (`brag.mp4`).

4.2 Major Engineering Breakdowns & Solved Hurdles

Hurdle 1: The 208-Monitor Draw Call Bottleneck

The Problem: In initial 3D prototypes, each desk monitor in the office was rendered as an independent Three.js mesh with a separate dynamic canvas texture. With 208 seated coworkers, this resulted in over 450 draw calls per frame, dropping framerates on standard laptops below 15 FPS.

The Solution: The team engineered a unified screen atlas subsystem (`screenPainters.ts`). All 208 individual role applications are rendered into a single 2048×2048 2D canvas sheet. All 208 monitor meshes were batched into a single static geometry sharing a single PBR material, reducing monitor draw calls from 208 down to exactly **1 draw call**. Framerates locked at a steady 60 FPS.

Hurdle 2: Upstream Reasoning Block Replay Rejection

The Problem: When cutting-edge reasoning models (Anthropic Claude 3.7 Sonnet with thinking blocks, Google Gemini 2.0 with thought signatures, DeepSeek R1 with reasoning content) emit tool calls, the model produces internal thinking tokens. If the client drops these thinking tokens during the subsequent tool-result round, upstream APIs reject the request with validation errors (e.g., Anthropic HTTP 400: *"Thinking block must be followed by matching signature"*).

The Solution: Axon's provider adapter (`providers.ts`) records the exact cryptographic thinking signatures and thought blocks during stream parsing and re-emits them byte-for-byte in the assistant turn history before appending the corresponding tool results.

Hurdle 3: Vulnerability Remediation (19 → 0 CVEs)

The Problem: Legacy dependencies inherited from early prototypes included vulnerabilities in transitive packages, most notably in `xlsx` (Prototype Pollution) and outdated Electron runtime binaries, yielding 19 security vulnerabilities during `npm audit`.

The Solution: The team executed a full dependency hardening cycle:

- Migrated desktop runtime to **Electron 44.4.1**.
- Upgraded build system to **electron-vite 5.0.0** and **Vite 7.3.6**.
- Completely excised `xlsx`, replacing it with the actively maintained **exceljs 4.4.0**.
- Implemented a package override for transitive `uuid` (v11.1.1).
- Result: **0 vulnerabilities** verified across all production and development dependencies.

4.3 Verification, Automated Testing & Smoke Test Protocols

Quality assurance in Axon is enforced across multiple tiers of automated verification:

Wire Format Protocol Tests

`TESTS/PROVIDERS.TEST.CJS`

Validates wire-format SSE parsing, tool serialization, thinking block replay, and error handling against mock HTTP transport without incurring paid API costs.

Live Desktop E2E Smoke Test

`TESTS/ELECTRON-SMOKE.CJS`

Spawns the built Electron application with an isolated temporary profile, connects to a loopback mock provider, executes a full streaming turn, asserts API keys arrive as `Bearer ...`, and verifies screenshot rendering.

Connectors Live Health Audit

`SCRIPTS/CONNECTORS-CHECK.MJS`

Audits all 39 catalog connector endpoints over the live network to verify endpoint reachability, TLS certificate validity, and OAuth discovery metadata.

Packaging & Binary Verification

`NPM RUN PACKAGE:DIR`

Executes `electron-builder` to generate unpacked Windows binaries (`dist/win-unpacked/Axon.exe`), validating clean zero-exit-code builds.

Chapter 5: What Axon Can Do — Operational Scenarios

5.1 Multi-Agent Collaborative Full-Stack Engineering

In complex software projects, single-agent workflows fail because a single prompt cannot simultaneously maintain deep frontend design sensibilities, backend concurrency constraints, and security standards. Axon's coworker ecosystem excels at cross-functional delegation:

Workflow Step	Active Coworker	Operational Action & System Invariant
1. Goal Definition	User → Chief of Staff	User requests: "Implement an audited OAuth sign-in flow with GitHub and Google." Chief of Staff breaks this down into frontend components, backend endpoints, and security audits.
2. Product Specs	Product Coach	Chief of Staff calls <code>ask_colleague("product-coach")</code> to establish acceptance criteria, error handling states, and session lifecycles.
3. UI & Design Tokens	Frontend Developer	Frontend Developer creates <code>LoginButtons.tsx</code> using the design system's semantic tokens, verifying keyboard focus rings and contrast ratios.
4. Backend Endpoints	Backend Developer	Backend Developer implements loopback redirect servers, PKCE challenge verifiers, and route handlers.
5. Security Review	Security Engineer	Chief of Staff invokes <code>ask_colleague("application-security-engineer")</code> to inspect the code for open redirect vulnerabilities and secret leakage.
6. Human Approval & Undo	User Review Surface	User inspects the consolidated unified diff in the Approval Card, reviews the safety summary, and clicks Approve. If needed, one-click Undo rolls back the changes immediately.

5.2 Automated Incident Triage via Sentry & GitHub MCP

When an unexpected runtime exception occurs in production, an on-call engineer can collaborate with Axon to investigate and remediate the issue within minutes:

1. **Error Ingestion:** The Site Reliability Engineer uses the Sentry connector to query recent unhandled exceptions:
`mcp_sentry_list_issues(project="core-api")`.
2. **Stack Trace Analysis:** The SRE extracts the stack trace and pinpointed line numbers, then asks the Python Developer colleague to inspect the relevant codebase files via `read_file`.
3. **Root Cause Identification:** The Python Developer discovers a null pointer dereference caused by a missing database column migration.
4. **Patch Proposal:** The developer proposes a patch. The user reviews the unified diff in an Approval Card and approves.
5. **Automated PR Submission:** With user confirmation, the coworker uses the GitHub connector (`mcp_github_create_pull_request`) to open a pull request with an explanatory description and issue link.

5.3 Air-Gapped Enterprise Knowledge Extraction

For legal firms, healthcare organizations, and financial institutions handling confidential data that cannot be uploaded to third-party vector databases:

- The user attaches multi-gigabyte regulatory PDFs, clinical trial DOCX files, and financial Excel models to a private workspace.
- Axon's isolated worker pool extracts the text in background sub-processes without leaking memory or sending a single byte to the internet.
- The Knowledge Librarian indexes the material using local BM25 ranking.
- The user queries the knowledge base against a local, private LLM (such as Llama 3 or DeepSeek running on Ollama over loopback HTTP).
- The model synthesizes findings with exact page citations, operating **100% offline** with zero third-party cloud data transmission.

Chapter 6: Security, Privacy & Boundary Enforcement

6.1 Defense-in-Depth Architecture

Axon rejects the conventional web-app security model in favor of an operating-system-level defense-in-depth architecture:

Renderer Process Isolation

ZERO NETWORK & FILESYSTEM ACCESS

The React renderer runs in a sandboxed context with `contextIsolation: true` and `nodeIntegration: false`. It cannot initiate raw socket connections, access the local disk, or read process memory. All actions must traverse the typed `window.axon` preload bridge.

IPC Sender Verification

ANTI-SPOOFING CHECKS

Every IPC handler in the main process verifies `event.senderFrame` to ensure that requests originate strictly from the trusted local application window rather than rogue webviews or injected scripts.

6.2 Filesystem Sandboxing & Jail Boundaries (`src/main/project.ts`)

When a conversation is granted access to a project folder, the path is strictly bounded:

- **Path Traversal Defense:** All relative paths are resolved and checked via `Project.safe()`. Any path containing `../` or resolving outside the designated project root is immediately rejected.
- **Symlink & Junction Escapes:** Symbolic links and NTFS junction points are resolved to their true physical targets. If a symlink points outside the project boundary, access is denied.
- **Sensitive File Protection:** Requests to read or write `.env` files, anything in `.git`, `.ssh`, `.aws`, `.docker` or `.kube`, SSH keys (`id_rsa` ...), `.npmrc`, `.git-credentials`, Terraform state and other credential files are blocked at the engine level, regardless of user approval. Names are compared without case (`.GIT` is `.git`), and Windows 8.3 short names and trailing dots are refused. Any file whose contents hold a private key or a live access token is never given to the model.
- **Folder Trust:** A folder opens restricted until the user trusts it: its instruction files stay out of the prompt and Axon runs no git there. In any folder, git runs without the repository's hooks or fsmonitor for anything a coworker does.

Chapter 7: Production Roadmap & Future Horizons

7.1 Immediate Release Blockers & Production Gates

The engineering gates are done in code: transactional SQLite storage with a write-ahead log and versioned migrations, parsers in an OS-sandboxed renderer, a signing and notarization pipeline that refuses unsigned releases, signed automatic updates with a minimum-version policy, Electron fuses, and CI on Windows, macOS and Linux. Before public general distribution, what remains needs accounts, certificates or people (see `README.md` and `docs/RELEASING.md`):

1. **Signing Credentials:** An Azure Trusted Signing account (or EV certificate) for Windows and an Apple Developer ID with notarization credentials, added as release secrets.
2. **OAuth App Registration:** The GitHub, Google (with OAuth verification and, for Gmail and Drive, a security assessment), Slack, HubSpot, Box and Asana apps, whose IDs are baked into release builds.
3. **Legal Review:** The privacy policy and end-user license agreement drafts, and a license (or removal) for the skill collection that ships only in development builds.
4. **Independent Security Review:** A penetration test of the agent permission layer and the app shell.

7.2 Mid-Term Enhancements & Enterprise Scaling

- **Multimodal Ingestion:** Direct image understanding, architectural diagram analysis, and audio transcription across coworker conversations.
- **Visual MCP Connector Builder:** An in-app GUI allowing non-developers to configure custom stdio and HTTP MCP servers with visual header and authentication testing.
- **Scheduled Autonomous Agent Workflows:** Permitting coworkers to execute unattended background tasks (e.g., nightly dependency vulnerability scans, morning PR briefings) with explicit time-bounded leases.

7.3 Long-Term Vision: The Shared Spatial Office

The ultimate vision for Axon extends beyond single-user desktop software into the **Multiplayer Shared Office**. In this future paradigm, distributed human engineering teams share a persistent 3D virtual campus with their AI coworkers. Human teammates walk their avatars into meeting rooms to collaborate with AI specialists, observe live coding monitors in the engineering district, and review company-wide operational velocity in real time.

Chapter 8: Technical Reference & Operations Manual

8.1 CLI & Build Tooling Reference

```
# Development Environment Startup
npm install          # Install all production and development dependencies
npm run dev          # Launch electron-vite HMR development server

# Validation & Testing Suite
npm run typecheck    # Run TypeScript compiler type-checking without emitting files
npm test             # Run unit test suite (providers, history, storage, MCP)
npm run connectors:check # Validate all 39 catalog connector endpoints against live network
npm run test:desktop  # Launch built application with loopback mock provider E2E smoke test

# Production Compilation & Packaging
npm run build         # Compile main, preload, and renderer bundles into out/
npm run package:dir   # Build unpacked executable in dist/win-unpacked/Axon.exe
npm run package       # Generate distribution installer (NSIS on Windows, DMG on Mac)

# Skill & Model Utilities
npm run skills:ingest # Re-ingest and compile catalog from skills.sources.json
npm run models:optimize # Process and compress 3D GLTF models for Three.js scene
```

8.2 Complete IPC Bridge API Reference (window.axon)

IPC Method	Parameters	Return Type	Functional Description
snapshot	—	Promise<SnapshotPayload>	Everything the window shows, without keys or connector secrets; the skill and role catalog comes once from <code>catalog()</code> .
chatSend	conversationId, text, attachmentIds	Promise<void>	Starts an assistant run; progress arrives as stream events, each new piece with its offset.
chatStop	conversationId	Promise<void>	Aborts the run and withdraws any approval it is waiting for.
toolApprove	{ requestId, approved, alwaysAllowSession? }	Promise<void>	Answers an approval card: once, or always in this conversation and folder.
revertChange	toolCallId	Promise<void>	Undoes a coworker's file write or memory update after a native confirmation.
providerSave	provider, key?	Promise<void>	Saves a provider; a key goes to the OS vault. A changed endpoint is confirmed natively.
providerTest	provider, key?	Promise<ProviderTestResult>	Checks up to ten models with a tiny request, without saving.
connectorAdd	catalogId	Promise<void>	Adds a catalog connector, signing in first when it needs to; a local one is confirmed natively.
auditList	query: AuditQuery	Promise<AuditEntry[]>	The activity log, newest first, filtered by conversation, coworker or kind.
auditExport	—	Promise<boolean>	Saves the whole hash-chained activity log as JSON or CSV.
listBackups	—	Promise<BackupSummary[]>	The restore points in <code>userData/backups/</code> , newest first.
restoreBackup	file	Promise<void>	Backs up the current state, restores the chosen restore point, and restarts Axon.
projectTrust	—	Promise<boolean>	Asks natively whether you trust the open folder.
permissionGrants / permissionRevoke	— / grantId	Promise<SessionGrant[]> / Promise<void>	The "always allow" grants in force, and taking one back.
diagnosticsExport	—	Promise<boolean>	Saves versions, set-up and the recent log, with keys and tokens removed.
updateCheck / updateInstall	—	Promise<UpdateState> / Promise<void>	Looks for an update of Axon; restarts into a downloaded one.

Conclusion of Dossier: This completes the end-to-end architectural, capability, inventory, historical, and operational documentation of Axon AI Studio (v0.2.0). All referenced files, schemas, and metrics reflect the verified state of the active project repository.